Distributed LRU Cache Manager

High-availability distributed systems leverage sophisticated caching layers to minimize latency and reduce pressure on primary database engines. A "Least Recently Used" (LRU) eviction policy is essential for managing memory constraints, ensuring that high-frequency data remains accessible while stale entries are systematically purged.

Implement a high-performance `LRUCache` class using `collections.OrderedDict`. The cache must maintain a strict capacity limit, promote accessed keys to the "Most Recently Used" (MRU) position, and automatically evict the "Least Recently Used" (LRU) item when the buffer is full.

Example 1 Input:

```
INIT 2 PUT 1 A PUT 2 B GET 1 PUT 3 C
```

Output:

```
A
```

Explanation: The cache is initialized with a capacity of 2. `1:A` and `2:B` are inserted. `GET 1` marks key 1 as the most recently used. When `PUT 3:C` is executed, key 2 (the oldest/LRU) is evicted to accommodate the new entry.

Example 2 Input:

```
INIT 2 PUT 1 A PUT 3 C GET 2
```

Output:

```
-1
```

Explanation: Key 2 does not exist in the current cache state, so the `GET` operation returns -1.

Function Parameter:

- capacity (int): The maximum number of key-value pairs allowed in the cache.

Returns

- Any: The value associated with the key if present; otherwise `-1`.

Constraints

- $1 \le capacity \le 10^4$
- Operations must achieve $O(1)$ average time complexity.
- Utilize `collections.OrderedDict` for internal state management.

Input Format for Custom Testing Input from stdin will be processed as follows: The first line contains `INIT` followed by the capacity. Subsequent lines contain `PUT key value` or `GET key`.

Sample Case 0 Sample Input

```
INIT 2 PUT 1 A PUT 2 B GET 1 PUT 3 C GET 2
```

Sample Output

```
A -1
```

Explanation Key 2 is evicted when Key 3 is added, as Key 1 was refreshed by the `GET` command.